

# Traffic Gotcha

A post-mortem of the `api.apexclasses.org` 502 / "CORS" outage — and why the bug was three layers away from where it looked.

| Service                                                | Symptom                              |  |            |
|--------------------------------------------------------|--------------------------------------|--|------------|
| api.apexclasses.org (Apex backend)                     | Browser CORS error + 502 Bad Gateway |  |            |
| Real cause                                             | Status                               |  | Date       |
| Stale hard-coded upstream IP in a Traefik static route | RESOLVED + HARDENED                  |  | 2026-06-09 |

**One-line takeaway:** A 502 strips CORS headers, so a dead backend masquerades as a CORS bug. The backend was healthy the whole time — the proxy was dialing a **dead IP** baked into a hand-written Traefik config file that never updated when the container was redeployed.

## 1 · What the user saw

The frontend at `apexclasses.org` could not load products. The browser console showed a textbook CORS failure:

```
Access to fetch at 'https://api.apexclasses.org/api/orders/products'
from origin 'https://apexclasses.org' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
GET https://api.apexclasses.org/api/orders/products net::ERR_FAILED 502 (Bad Gateway)
Fetch products error: TypeError: Failed to fetch
```

Two clues sat right next to each other and most people read only the first:

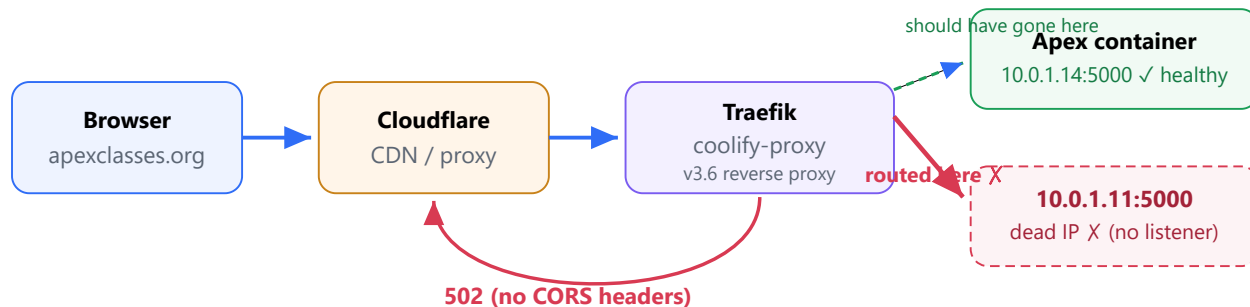
- **"CORS policy ... No Access-Control-Allow-Origin"** — looks like a backend CORS misconfiguration.
- **"502 (Bad Gateway)"** — the *actual* signal. The request never reached a working app.

### THE GOTCHA

When a reverse proxy returns a **502**, it returns its own error page — which carries **no CORS headers**. The browser then fails its CORS check and reports a CORS error. **CORS was a symptom, not the disease**. Chasing the CORS message means debugging the wrong layer for hours.

## 2 · The system, end to end

Requests cross four layers before reaching the Node app. The bug lived at the proxy.



The request path. Traefik matched the route but dialed a dead upstream → 502 propagated back to the browser as a CORS failure.

### 3 · The investigation — what we tried and what it told us

Each step either fixed nothing or eliminated a suspect. The value was in *narrowing*, not in any single lucky guess.

| #  | Hypothesis / action                                                                                                              | Result                                                    | What it ruled out                                                                                    |
|----|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1  | Read backend CORS config ( <code>app.js</code> ) — is the origin allow-list wrong?                                               | CONFIG CORRECT                                            | Not a CORS code bug — apexclasses.org was already allow-listed.                                      |
| 2  | Probe live <code>/health</code> over the public URL.                                                                             | 502                                                       | Even the trivial health route 502s → whole app unreachable, not one endpoint.                        |
| 3  | Check app status in Coolify; read container logs.                                                                                | Status <code>exited</code> ; logs end at 🚀 Server running | App boots cleanly (DB + socket OK) then “dies” with no error → looked like a crash.                  |
| 4  | Restart, then full force-redeploy.                                                                                               | STILL 502                                                 | Not a one-off bad container.                                                                         |
| 5  | OOM theory — remove other heavy containers (voice agents) to free RAM.                                                           | NO CHANGE                                                 | Not memory pressure / OOM.                                                                           |
| 6  | Code review of post-startup crash paths (pg pool, socket.io).                                                                    | CLEAN                                                     | <code>pool.on('error')</code> exists; no timers/unawaited throws → no silent crash.                  |
| 7  | <b>Get into the host:</b> <code>docker inspect</code> exit code + <code>docker logs</code> + <code>wget</code> to the container. | Container UP, <code>/health</code> = 200 internally       | <b>Turning point.</b> The app was never down. The problem is the <i>proxy</i> → <i>app</i> hop.      |
| 8  | Test origin directly (Host header, bypass Cloudflare) on :80 and :443.                                                           | :80 → 302, :443 → 502                                     | 502 is Traefik-side, TLS is fine — Traefik can't reach the backend.                                  |
| 9  | Confirm network reachability + restart Traefik.                                                                                  | reachable; restart<br>DIDN'T FIX                          | Not a stale in-memory routing table.                                                                 |
| 10 | Discover Traefik's <b>file provider</b> and read <code>production.yaml</code> .                                                  | ROOT CAUSE FOUND                                          | A hand-written route hard-codes <code>apex-backend</code> → <code>10.0.1.11:5000</code> — a dead IP. |

#### WHY "EXITED" WAS A RED HERRING

Coolify's UI labeled the app `exited`, but `docker inspect` showed `ExitCode=0` with `FinishedAt=0001-01-01T00:00:00Z` — Docker's **zero value**, meaning the container **never actually stopped**. The status label was wrong; the container was running and listening the entire time.

## 4 · How we recognized the real problem

---

Three observations, stacked, pointed away from the app and at the proxy:

1. **Internal health = 200.** From another container on the same Docker network:

```
# run from a neighbouring container
$ wget -qO- http://10.0.1.14:5000/health
{"status":"ok, From Apex Backend","time":"2026-06-09T11:22:27Z"}
```

The backend answers perfectly. So the failure is *between* the proxy and the app.

2. **Origin 502 even bypassing Cloudflare.** Hitting Traefik directly with a forged Host header still 502'd on HTTPS — so Cloudflare was innocent; Traefik itself could not complete the upstream dial.
3. **Traefik had a *file* provider, not just Docker labels.** The static file pinned the route to a literal IP:

```
# /data/coolify/proxy/dynamic/production.yaml (mounted into coolify-proxy)
services:
  apex-backend:
    loadBalancer:
      servers:
        - url: http://10.0.1.11:5000 # <-- hard-coded, now DEAD
```

**The smoking gun.** The live container had been redeployed many times; each redeploy gave it a **new IP** (now 10.0.1.14 ). The static file still said 10.0.1.11 — an address that, after all the churn, had been reassigned to the **Traefik proxy itself** (which doesn't listen on :5000). Traefik was effectively dialing a dead end and returning 502.

### Why this static route “won” over the correct one

Coolify *also* generates a correct, auto-updating Docker-label route for the same domain. But Traefik resolves conflicts by **rule specificity**: the file's rule `Host(apex.rohithuge.online) || Host(api.apexclasses.org)` is longer than the label's single-host rule, so the **stale file route took priority** and shadowed the good one.

## 5 · The fix

---

### Step 1 — Immediate (stop the bleeding)

Point the static route at the live container IP. Traefik's file-watch reloads automatically — no restart needed.

```
$ cp production.yaml production.yaml.bak-20260609 # backup first
$ sed -i 's/10\.0\.1\.11/10.0.1.14/g' production.yaml # dead IP -> live IP
# ~5s later
$ curl -sk -H 'Host: api.apexclasses.org' https://localhost/health
origin -> 200
```

### Step 2 — Durable (so it can't recur)

A hard-coded IP breaks again on the *next* redeploy. The real fix: delete the apex stanza from the static file and let Coolify's **Docker-label route** own the domain — it tracks the container automatically.

**Safety pre-check** (so we don't 404): verify the label route is complete before removing the file route —

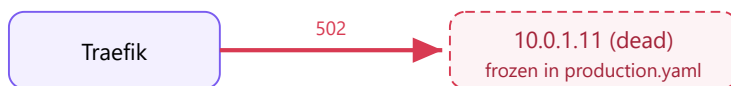
```
traefik.enable=true
routers.https-0-...rule = Host(`api.apexclasses.org`)
routers.https-0-...tls = true    tls.centresolver = letsencrypt
services.https-0-...loadbalancer.server.port = 5000    ✓ all present
```

Then removed the apex router + service from `production.yaml` (everything else untouched), Traefik reloaded, and:

```
$ curl -sk -H 'Host: api.apexclasses.org' https://localhost/health
{"status":"ok, From Apex Backend", ...} http code: 200

# public, through Cloudflare
/health          -> 200
/api/orders/products -> 200
CORS preflight   -> 204  Access-Control-Allow-Origin: https://apexclasses.org
```

#### BEFORE — static file route (hard-coded IP)



#### AFTER — Docker-label route (auto-tracked)



Redeploy → new IP?  
**Route follows automatically ✓**

Before vs after. Hard-coded IPs rot on every redeploy; label-based routing re-resolves the upstream each time.

## 6 · Scope check — what we deliberately did *not* touch

The same static file also routes `api.jeetexam.in`, `upload.*`, `auth.*` and `db.*`. A discovery pass showed these domains have **no Docker-label route** — they *depend* on the static file. Removing them would have 404'd live services, so we left them as-is (their IPs were verified current). Migrating them properly means adding each domain inside its Coolify app first — a separate, per-service change on production.

## 7 · Lessons (the part worth keeping)

502 ≠ CORS

READ THE SECOND ERROR TOO

UI STATUS CAN LIE

HARD-CODED IPS ROT

VERIFY AT EVERY HOP

- **A CORS error with a 502 next to it is a transport failure, not a CORS bug.** The missing `Access-Control-Allow-Origin` header is what an error page looks like — fix the 502 and CORS fixes itself.
- **Don't trust the dashboard's status word.** `exited` in the UI vs `FinishedAt=0001-01-01` in `docker inspect` — go to the source of truth.

- **Bisect by layer.** The breakthrough was one command: `wget` the container directly. App-200 + public-502 instantly localizes the bug to the proxy hop.
- **Prefer dynamic service discovery over hard-coded IPs.** Container IPs are ephemeral; any config that pins one is a time bomb that detonates on the next deploy.
- **Beware “shadow” config.** Traefik merged a hand-written *file* provider with auto-generated *labels*; the stale file silently out-prioritized the correct route. Know every provider feeding your proxy.

## 8 · Quick reference — diagnostic playbook

---

```
# 1. Is it the app or the path to the app? Hit the container directly.
docker exec <neighbour> wget -qO- http://<container-ip>:5000/health

# 2. Is it the proxy or Cloudflare? Forge the Host header on the origin.
curl -sk -H 'Host: api.example.com' https://localhost/health -w '%{http_code}\n'

# 3. Is the container really dead? Trust inspect, not the dashboard.
docker inspect <c> --format 'Exit={{.State.ExitCode}} OOM={{.State.OOMKilled}} Finished={{.State.FinishedAt}}'

# 4. What routes does Traefik actually have? Check BOTH providers.
docker inspect coolify-proxy --format '{{json .Args}}'      # find --providers.file.directory
grep -rl <domain> /data/coolify/proxy/dynamic/              # the shadow config
```